GitHub Copilot CLI — Configuration Report

Prepared for **Murpheux** · 28 September 2026 · Verified against **Copilot CLI 1.0.61** (installed on this workstation)
Folder: `~/agent-configs/github-copilot-cli/` → installs to `~/.copilot/` (shared with VS Code's Copilot harness)

1. Summary

The Copilot CLI has everything the design needs:

- **custom agents with tool restrictions**, so the router literally cannot edit or run shell;
- **per-subagent model and effort tiers** (`subagents.agents`);
- personal instructions and skills;
- **fail-closed preToolUse hooks**;
- a sandbox, and optional **managed deny/ask/allow rules**.

Because `~/.copilot` is also read by VS Code, instructions, skills and hooks are identical to the `vscode/` folder. Agents are target-scoped (`github-copilot`) so VS Code doesn't list them.

8 custom agents	**11** tiered subagents	**33** skills	**7** shell entry points	**4** MCP + built-in GitHub

2. Sources used

- Local ground truth: `copilot help config|permissions|providers|environment|commands|billing|limits` and `copilot mcp|skill|plugin --help`.
- `github/docs`: CLI configuration-directory reference, hooks reference, custom-agents configuration.
- **github/awesome-copilot** and **awesome-copilot.github.com** (built into the CLI as a plugin marketplace).
- The dedicated research agent stalled, so these primary sources were read directly instead.

3. What was built

File	Purpose
`settings.fragment.jsonc` → merged into `~/.copilot/settings.json`	default model/effort; per-agent tiers for the 7 custom agents plus built-ins (explore, task, code-review, general-purpose); maxConcurrency 4 / maxDepth 2; sandbox with credential paths denied; `deniedUrls` exfiltration list; `remote: off`; `disableBypassPermissionsMode: allow-auto-only`; spend footer
`instructions/murpheux.instructions.md`	global instructions (`applyTo: "**"`), loaded by the CLI and both VS Code harnesses
`agents/*.agent.md`	router (gpt-5-mini; tools agent/read/search/web/todo), explorer, reviewer, planner, developer, sre, architect, security
`skills/`	20 portable + 13 workflow skills (built-in `/review` `/plan` `/pr` used instead of duplicates)
`hooks/guard.json` + `guard.py`	`preToolUse` guard (bash and PowerShell variants); exit 2 denies, crashes fail closed
`mcp-config.json`	context7, mslearn, awsdocs, terraform (GitHub MCP is built in)

`copilot-aliases.sh/.ps1`	`cop` (router), `cop-dev`, `cop-sre`, `cop-plan`, `cop-sec`, `cop-cheap`, `cop-cap` — all with `--secret-env-vars` and `--deny-tool shell(git push)`
`managed/managed-settings.json`	optional root-owned policy: deny/ask/allow rules the model can't bypass (`install.sh --managed`)

4. Agent tiers

Agent	Model · effort	Tools
router	gpt-5-mini	agent, read, search, web, todo
explorer · built-in explore/task	gpt-5-mini · low	read-only · builds/tests
reviewer · built-in code-review	claude-haiku-4.5 · medium	read, search, execute
planner	gpt-5.5 · medium	read-only + agent
developer · sre · general-purpose	claude-sonnet-5 · medium	all · ops
architect	gpt-5.5 · high	read + edit docs
security	claude-opus-4.8 · high	read + scanners

Model ids follow the v1.0.61 catalog (`copilot help config`); the picker shows per-token prices. Adjust them in one place, `subagents.agents`.

5. Cost levers

- A cheap router, plus cheap built-in explore/task/code-review, so the most frequent helper calls don't run on the default model.
- AI-credit visibility in the footer; `/usage` and `/context`; `cop-cap` sets `--max-ai-credits` (a soft cap, minimum 30).
- `continueOnAutoMode: false`: no silent model switches.
- Autopilot is still capped by `--max-autopilot-continues` (default 5).

6. Guardrails

1. **Tool restriction**: the router has no edit or execute tools.
2. **Guard hook**: the shared 33 command rules and 18 protected paths. The CLI treats hook crashes and non-zero exits as a deny (fail closed); timeouts fail open.
3. **Sandbox** (macOS/Linux): `~/.ssh`, `~/.aws`, `~/.gnupg`, `~/.config/op`, kube/docker configs and AI-tool credential files are denied. On Windows the installer drops `deniedPaths`, because that backend rejects them.
4. **Secrets**: `--secret-env-vars` strips and redacts gateway, API and cloud tokens; `deniedUrls` blocks paste and webhook exfiltration hosts.
5. **No bypass**: allow-all flags are refused, and `remote: off` keeps sessions local.
6. **Managed policy** (optional): real deny/ask/allow rules. Note that once any list is set, unmatched operations ask, which is why reads and read-only commands are allow-listed.

7. From awesome-copilot

- **Install** (built-in marketplace): `csharp-dotnet-development`, `aws-cloud-development`, `azure-cloud-development`, `devops-oncall`, `structured-autonomy` ("premium planning, thrifty implementation").
- **Adopted natively**:
  - *rug-orchestrator* (pure-coordinator router) → router agent;

    - *tool-guardian* and *secrets-scanner* hooks → guard.py;
    - *terraform-iac-reviewer* and *se-security-reviewer* → security agent and `iac-review`;
    - *platform-sre-kubernetes* → sre;
    - draw.io, threat-model and ADR skills → shared skills.
- Large always-on instructions (e.g. `security-and-owasp`, 1,088 lines) were kept as on-demand skills to avoid a per-request token cost.

8. Gotchas handled

- User-level `settings.json` has no allow/deny arrays; only managed settings do. So persistent denial relies on hooks, the sandbox, aliases and the optional managed file.
- User settings moved from `config.json` to `settings.json` (JSONC). The installer *merges* rather than overwriting.
- `~/.agents/skills` (Codex) is also read by the CLI, so the installer skips identical duplicates.
- `--deny-tool shell(…)` matches first-level subcommands only for git/gh; everything else is covered by the hook.

9. Validation

Configs parse. The guard's 70 self-tests passed, including CLI payloads (`toolName` / `toolArgs`) with deny → exit 2. The installer smoke test merged settings while keeping an existing key (`theme`) and installed 8 agents, 33 skills and the hook. Both `.ps1` scripts parse cleanly.

10. Install

```
cd ~/agent-configs/github-copilot-cli && DRY_RUN=1 ./install.sh && ./install.sh     # macOS / WSL
echo 'source ~/.copilot/copilot-aliases.sh' >> ~/.zshrc && cop
./install.sh --managed                                                             # optional hard policy (sudo)
# Windows: copy the folder locally, then  pwsh -File install.ps1  (UNC scripts are blocked by RemoteSigned)
```